



Práctica 1

Seguridad en el Desarrollo de Aplicaciones

*División de Tecnologías de la Automatización e
Información (DTAI) Área Desarrollo y Gestión Software*

Grupo:

IDGS17

Alumna:

Arlet Roxana Herrera Vázquez

Matrícula:

2024171035

Fecha de entrega:

19/05/2026

ÍNDICE

Análisis de OWASP Top 10 2023.....	3
Tabla comparativa – OWASP Top 10 2023.....	4
Casos reales por vulnerabilidad.....	6
1. Broken Object Level Authorization (BOLA).....	6
2. Broken Authentication.....	7
3. Broken Object Property Level Authorization.....	8
4. Unrestricted Resource Consumption.....	9
5. Broken Function Level Authorization.....	10
6. Unrestricted Access to Sensitive Business Flows.....	11
7. Server Side Request Forgery (SSRF).....	12
8. Security Misconfiguration.....	13
9. Improper Inventory Management.....	14
10. Unsafe Consumption of APIs.....	15
Código vulnerable vs seguro.....	16
1. SQL Injection.....	16
2. Broken Authentication.....	17
3. SSRF.....	18
Ranking personal – 5 vulnerabilidades más críticas.....	19
1. Broken Access Control.....	19
2. Broken Authentication.....	19
3. SSRF.....	19
4. Security Misconfiguration.....	19
5. Unsafe Consumption of APIs.....	19
Conclusión.....	20
Referencias.....	21

Análisis de OWASP Top 10 2023

Introducción

El OWASP Top 10 es una de las referencias más importantes en ciberseguridad para identificar las vulnerabilidades más críticas en aplicaciones web y APIs. La organización OWASP (Open Worldwide Application Security Project) publica periódicamente listados con los riesgos más relevantes basados en incidentes reales, investigaciones y reportes de seguridad.

En 2023, OWASP actualizó especialmente el enfoque de seguridad para APIs, destacando riesgos relacionados con autorización, autenticación, consumo inseguro de APIs y errores de configuración. Estas vulnerabilidades afectan directamente a empresas, gobiernos, plataformas de comercio electrónico y servicios financieros.

Tabla comparativa – OWASP

Top 10 2023

Vulnerabilidad	Descripción	Ejemplo de ataque	Impacto	Mitigación
API1: Broken Object Level Authorization (BOLA)	El sistema no valida correctamente si un usuario tiene permisos para acceder a un recurso específico.	Un atacante cambia el ID de una cuenta bancaria en la URL para acceder a información ajena.	Robo de datos sensibles y acceso no autorizado.	Validar autorizaciones en cada solicitud y aplicar controles de acceso estrictos.
API2: Broken Authentication	Fallas en autenticación que permiten robo de sesiones o credenciales.	Ataques de fuerza bruta contra formularios de login.	Compromiso de cuentas y robo de identidad.	Implementar MFA, políticas de contraseñas y tokens seguros.
API3: Broken Object Property Level Authorization	Exposición o modificación indebida de propiedades internas de objetos.	Un usuario modifica el rol "admin" mediante una petición JSON manipulada.	Escalamiento de privilegios y corrupción de datos.	Filtrar atributos permitidos y validar entradas del usuario.
API4: Unrestricted Resource Consumption	El sistema no limita correctamente el uso de recursos.	Miles de peticiones automáticas generan denegación de servicio.	Saturación del servidor y caída del sistema.	Implementar rate limiting y cuotas de consumo.

API5: Broken Function Level Authorization	Usuarios acceden a funciones administrativas sin autorización.	Un usuario común accede a endpoints de administrador.	Modificación de datos críticos y control del sistema.	Verificar permisos según roles y privilegios.
API6: Unrestricted Access to Sensitive Business Flows	Flujos críticos del negocio pueden ser abusados mediante automatización.	Bots compran boletos masivamente antes que usuarios legítimos.	Pérdidas económicas y abuso de servicios.	Detección de bots y validaciones de comportamiento.
API7: Server Side Request Forgery (SSRF)	El servidor realiza solicitudes a recursos internos manipulados por el atacante.	Un atacante obliga al servidor a consultar direcciones internas de AWS.	Exposición de infraestructura interna y datos confidenciales.	Validar URLs, restringir accesos internos y segmentar redes.
API8: Security Misconfiguration	Configuraciones inseguras en servidores, frameworks o servicios.	Consolas administrativas expuestas públicamente.	Exposición de datos y acceso remoto no autorizado.	Configuración segura, auditorías y hardening de servidores.
API9: Improper Inventory Management	APIs antiguas o no documentadas permanecen expuestas.	Un endpoint obsoleto permite acceso sin autenticación.	Vulnerabilidades invisibles y fuga de información.	Mantener inventario actualizado y eliminar APIs no utilizadas.
API10: Unsafe Consumption of APIs	La aplicación confía excesivamente en APIs externas inseguras.	Una API externa comprometida inyecta datos maliciosos.	Infección de sistemas y corrupción de datos.	Validar respuestas externas y aplicar sanitización.

Casos reales por vulnerabilidad

1. Broken Object Level Authorization (BOLA)

Caso 1 – Facebook (2019)

Empresa: Facebook

Investigadores descubrieron que ciertos endpoints permitían modificar identificadores de usuario en solicitudes API. Esto permitía acceder a datos de perfiles ajenos.

Cómo fue explotada:

Los atacantes alteraban IDs dentro de solicitudes HTTP para obtener información de otros usuarios.

Caso 2 – Peloton (2021)

Empresa: Peloton

Una API pública exponía información privada de usuarios y celebridades.

Cómo fue explotada:

La API permitía acceder a perfiles privados simplemente modificando parámetros de identificación.

2. Broken Authentication

Caso 1 – Uber (2016)

Empresa: Uber

Los atacantes obtuvieron credenciales almacenadas incorrectamente en GitHub.

Cómo fue explotada:

Con las credenciales comprometidas pudieron acceder a sistemas internos y datos de millones de usuarios.

Caso 2 – Twitter (2020)

Empresa: Twitter

Un ataque de ingeniería social comprometió cuentas verificadas.

Cómo fue explotada:

Los atacantes engañaron a empleados para obtener acceso a herramientas administrativas internas.

3. Broken Object Property Level Authorization

Caso 1 – GitLab (2021)

Empresa: GitLab

Una vulnerabilidad permitió modificar atributos sensibles mediante peticiones manipuladas.

Cómo fue explotada:

Usuarios podían cambiar propiedades internas y obtener privilegios elevados.

Caso 2 – Shopify Apps (2022)

Empresa: Shopify

Algunas aplicaciones permitían modificar campos ocultos enviados desde el frontend.

Cómo fue explotada:

Los atacantes manipulaban parámetros JSON para alterar permisos.

4. Unrestricted Resource Consumption

Caso 1 – GitHub (2018)

Empresa: GitHub

GitHub sufrió uno de los mayores ataques DDoS registrados.

Cómo fue explotada:

Los atacantes enviaron tráfico masivo mediante servidores Memcached expuestos.

Caso 2 – Dyn DNS (2016)

Empresa: Dyn

Servicios como Netflix y Twitter fueron afectados.

Cómo fue explotada:

La botnet Mirai generó millones de solicitudes usando dispositivos IoT comprometidos.

5. Broken Function Level Authorization

Caso 1 – T-Mobile (2022)

Empresa: T-Mobile

Endpoints administrativos podían ser accedidos con validaciones deficientes.

Cómo fue explotada:

Los atacantes descubrieron funciones internas accesibles sin privilegios adecuados.

Caso 2 – Panera Bread (2018)

Empresa: Panera Bread

Una API exponía información de clientes sin controles adecuados.

Cómo fue explotada:

Los investigadores accedieron a datos personales usando endpoints inseguros.

6. Unrestricted Access to Sensitive Business Flows

Caso 1 – Ticketmaster (múltiples años)

Empresa: Ticketmaster

Bots compraban entradas automáticamente antes de usuarios legítimos.

Cómo fue explotada:

Scripts automatizados abusaban del flujo de compra.

Caso 2 – Nike Sneakers App (2020)

Empresa: Nike

Bots adquirían ediciones limitadas rápidamente.

Cómo fue explotada:

Automatización masiva del proceso de compra mediante APIs.

7. Server Side Request Forgery (SSRF)

Caso 1 – Capital One (2019)

Empresa: Capital One

Una vulnerabilidad SSRF permitió acceder a recursos internos en AWS.

Cómo fue explotada:

La atacante obtuvo credenciales temporales del servicio EC2 metadata.

Caso 2 – Microsoft Exchange (2021)

Empresa: Microsoft

Vulnerabilidades ProxyLogon incluían SSRF.

Cómo fue explotada:

Los atacantes ejecutaban solicitudes internas para comprometer servidores Exchange.

8. Security Misconfiguration

Caso 1 – MongoDB Expuesto (2017)

Empresa: múltiples organizaciones

Miles de bases de datos MongoDB quedaron accesibles públicamente.

Cómo fue explotada:

Administradores dejaron configuraciones por defecto sin autenticación.

Caso 2 – Equifax (2017)

Empresa: Equifax

Un componente vulnerable no fue actualizado.

Cómo fue explotada:

La falta de parches permitió acceso a datos de millones de personas.

9. Improper Inventory Management

Caso 1 – T-Mobile API (2023)

Empresa: T-Mobile

APIs antiguas continuaban activas sin monitoreo.

Cómo fue explotada:

Los atacantes localizaron endpoints obsoletos expuestos públicamente.

Caso 2 – Facebook Internal APIs

Empresa: Facebook

Algunas APIs internas fueron expuestas accidentalmente.

Cómo fue explotada:

Fallas de documentación e inventario dejaron endpoints visibles.

10. Unsafe Consumption of APIs

Caso 1 – SolarWinds (2020)

Empresa: SolarWinds

Dependencias comprometidas distribuyeron código malicioso.

Cómo fue explotada:

La cadena de suministro permitió introducir malware en clientes.

Caso 2 – Log4Shell (2021)

Empresa: múltiples organizaciones

Aplicaciones consumían bibliotecas vulnerables.

Cómo fue explotada:

La librería Log4j permitía ejecución remota de código.

Código vulnerable vs seguro

1. SQL Injection

Código vulnerable

```
const query = "SELECT * FROM usuarios WHERE usuario='" + user + "' AND  
password='" + pass + "'";  
db.execute(query);
```

Problema:

El atacante puede insertar comandos SQL maliciosos.

Código seguro

```
const query = "SELECT * FROM usuarios WHERE usuario=? AND password=?";  
db.execute(query, [user, pass]);
```

Mitigación:

Uso de consultas parametrizadas.

2. Broken Authentication

Código vulnerable

```
if(password === "admin123"){  
  acceso = true;  
}
```

Problema:

Contraseña hardcodeada y sin cifrado.

Código seguro

```
const bcrypt = require('bcrypt');  
const valido = await bcrypt.compare(password, hashGuardado);  
if(valido){  
  acceso = true;  
}
```

Mitigación:

Uso de hashes seguros y validación robusta.

3. SSRF

Código vulnerable

```
app.get('/fetch', async(req,res)=>{  
  const response = await axios.get(req.query.url);  
  res.send(response.data);  
});
```

Problema:

El atacante puede acceder a recursos internos.

Código seguro

```
const dominiosPermitidos = ['api.miempresa.com'];  
  
app.get('/fetch', async(req,res)=>{  
  const url = new URL(req.query.url);  
  
  if(!dominiosPermitidos.includes(url.hostname)){  
    return res.status(403).send('URL no permitida');  
  }  
  
  const response = await axios.get(req.query.url);  
  res.send(response.data);  
});
```

Mitigación:

Validar dominios y restringir accesos internos.

Ranking personal – 5 vulnerabilidades más críticas

1. Broken Access Control

Es la más crítica porque permite acceso directo a recursos restringidos. Un fallo de autorización puede comprometer información financiera, médica o gubernamental.

2. Broken Authentication

La autenticación deficiente facilita secuestro de cuentas y robo de identidad. Es una de las puertas de entrada más frecuentes.

3. SSRF

Esta vulnerabilidad puede comprometer infraestructura cloud completa, especialmente en AWS y Azure.

4. Security Misconfiguration

Muchas organizaciones dejan configuraciones por defecto, puertos abiertos o servicios inseguros. Es extremadamente común.

5. Unsafe Consumption of APIs

Actualmente las aplicaciones dependen de múltiples servicios externos. Un proveedor comprometido puede afectar miles de sistemas.

Conclusión

El OWASP Top 10 representa una guía esencial para comprender las amenazas más críticas en las aplicaciones. Los casos previamente analizados demuestran que incluso compañías grandes como Facebook, Uber, Capital One o Microsoft han sido vulnerables a errores incluidos en OWASP. Esto evidencia que la ciberseguridad no depende únicamente de la tecnología utilizada, sino también de procesos adecuados, monitoreo constante y capacitación continua.

En conclusión, el OWASP Top 10 es una herramienta fundamental para mejorar la seguridad de aplicaciones y APIs. Su implementación reduce riesgos, protege información sensible y fortalece la confianza de usuarios y clientes en los sistemas digitales modernos.

Referencias

- Open Worldwide Application Security Project. (2023). *OWASP API Security Top 10 2023*. OWASP Foundation. Recuperado de OWASP API Security Top 10 2023
- Open Worldwide Application Security Project. (s.f.). *Fundación OWASP*. Recuperado de OWASP Foundation
- Microsoft. (2021). *Guía para clientes sobre vulnerabilidades en servidores Microsoft Exchange locales*. Recuperado de Microsoft Security Blog
- Capital One. (2019). *Incidente de seguridad de datos de Capital One*. Recuperado de Capital One Security Incident Information
- Uber. (2017). *Incidente de seguridad de datos de 2016*. Recuperado de Uber Newsroom
- GitHub. (2018). *Reporte del incidente DDoS del 28 de febrero*. Recuperado de GitHub Engineering Blog
- Equifax. (2017). *Incidente de ciberseguridad e información importante para consumidores*. Recuperado de Equifax Security Incident Response
- Twitter. (2020). *Actualización sobre nuestro incidente de seguridad*. Recuperado de Twitter Blog
- SolarWinds. (2020). *Aviso de seguridad*. Recuperado de SolarWinds Security Advisory
- MongoDB. (2017). *Documentación de seguridad de MongoDB*. Recuperado de MongoDB Security Documentation